



Algorithmen und Datenstrukturen

Leon Johann Brettin



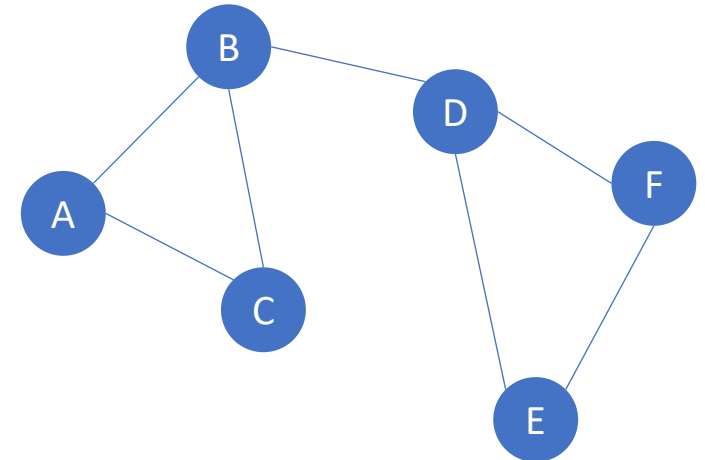
Algorithmen und Datenstrukturen

16. Graphen – Algorithmen auf Graphen



Spannbäume

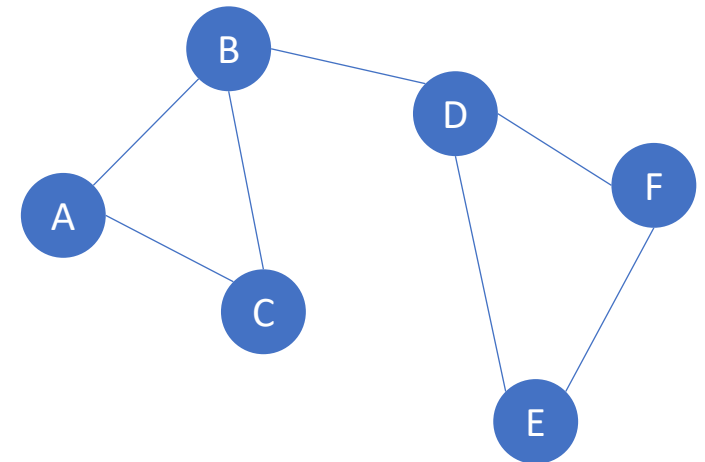
- Ein **Spannbaum** eines Graphen G ist ein **Baum**, der alle Knoten von G enthält und nur Kanten aus G enthält.
 - verschiedenste Spannbäume!
- **Tiefensuche**
 - Verfolge Kanten, bis du zu einem schon besuchten Knoten kommst.
 - Dann gehe einen Knoten zurück im Baum und suche weiter.
 - Bis du wieder am Anfang bist.





Breitensuche

- Erzeuge eine Warteschlange.
- Markiere zunächst alle Kanten, die vom aktuellen Knoten zu noch nicht markierten Knoten führen, und packe diese Nachbarn in die Warteschlange.
- Dann gehe zum nächsten Knoten in der Warteschlange.
- Bis die Schlange leer ist.





Breitendurchlauf – Breitensuche

- Knoten eines Graphen nach Entfernung vom Startknoten anordnen / durchlaufen
- Realisierung für ungerichteten Graphen
 - Warteschlange als Zwischenspeicher
 - Farbmarkierungen geben Status der Knoten an
 - weiß: unbearbeitet
 - grau: in Bearbeitung
 - schwarz: abgearbeitet
- pro Knoten wird Entfernung zum Startknoten berechnet, sowie der Knoten, von dem aus dieser Knoten erreicht wurde
- Diese Vorgängerkanten π bilden einen aufspannenden Baum mit Startknoten als Wurzel (Kanten π für predecessor)



Breitendurchlauf – Breitensuche (breadth-first search)

```
algorithm BFS(G, s)
  Eingabe: ein Graph G, ein Startknoten s des Graphen aus V[G]
  foreach Knoten u aus V[G] \ s do
    farbe[u] := weiß; d[u] := unendlich; pi[u] := null
  od
  farbe[s] := grau; d[s] := 0; pi[s] := null;
  Q = new Queue; Q.enter(s);
  while ! Q.isEmpty() do
    u := Q.leave();
    foreach v in „Zielknoten ausgehender Kanten“ von u do
      if farbe[v] = weiß then
        farbe[v] := grau; d[v] := d[u] + 1; pi[v] := u;
        Q.enter(v)
      fi
    od
    farbe[u] := schwarz
  od
```

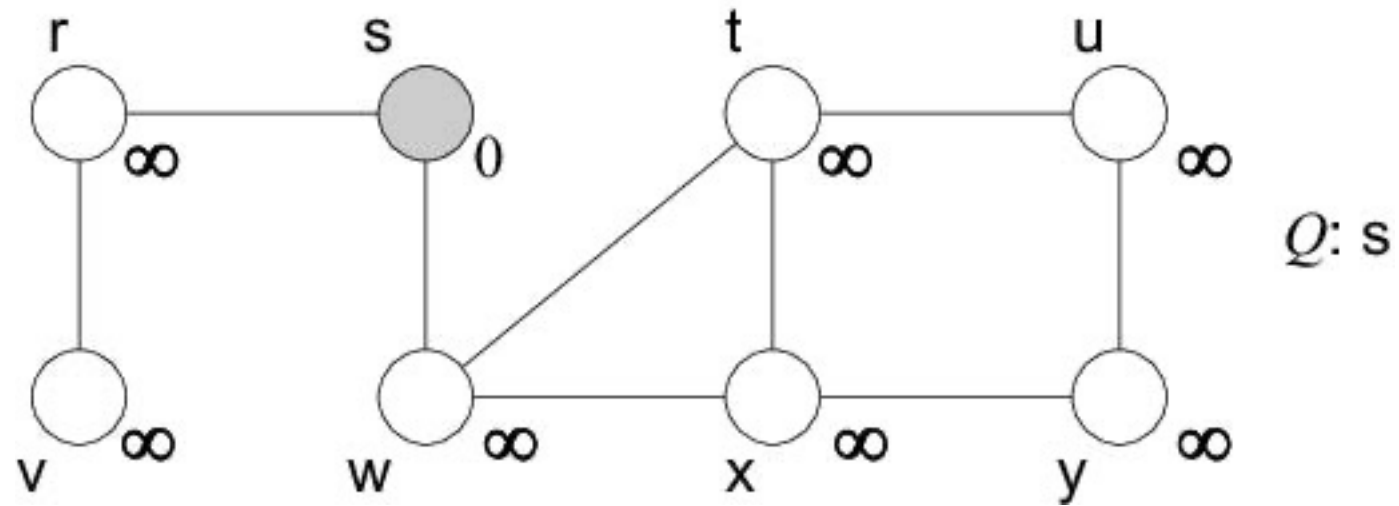


Breitendurchlauf – Initialisierung

- Startknoten
 - in Warteschlange einfügen
 - Farbe grau
 - Entfernung 0
 - Wurzel des aufspannenden Baums, deshalb keinen Vorgänger in π
- alle anderen Knoten
 - Entfernung unendlich
 - Farbe weiß
 - kein Vorgänger in π , da noch nicht in aufspannenden Baum eingefügt.



Breitendurchlauf – Beispiel

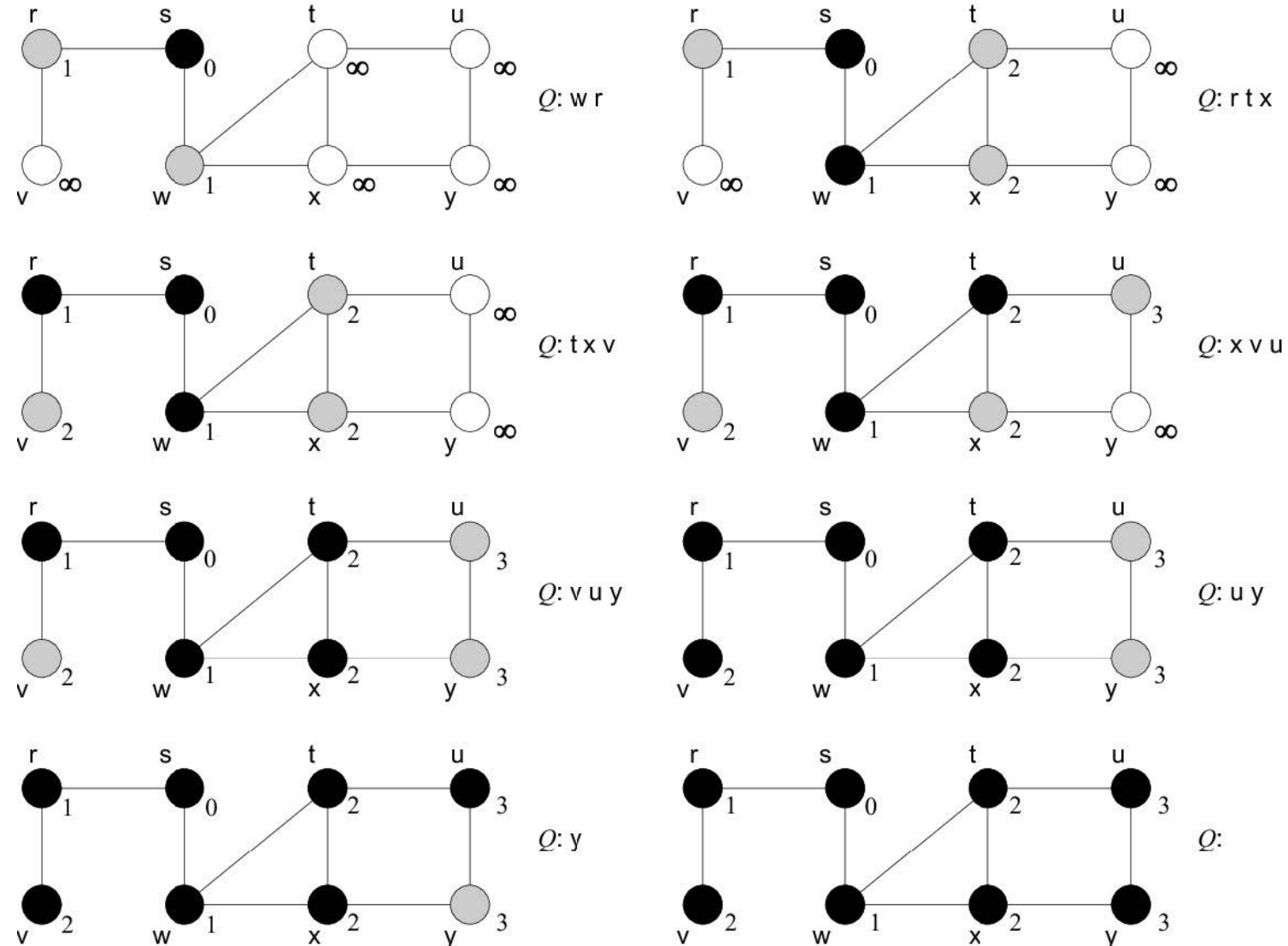




Breitendurchlauf – Einzelschritt

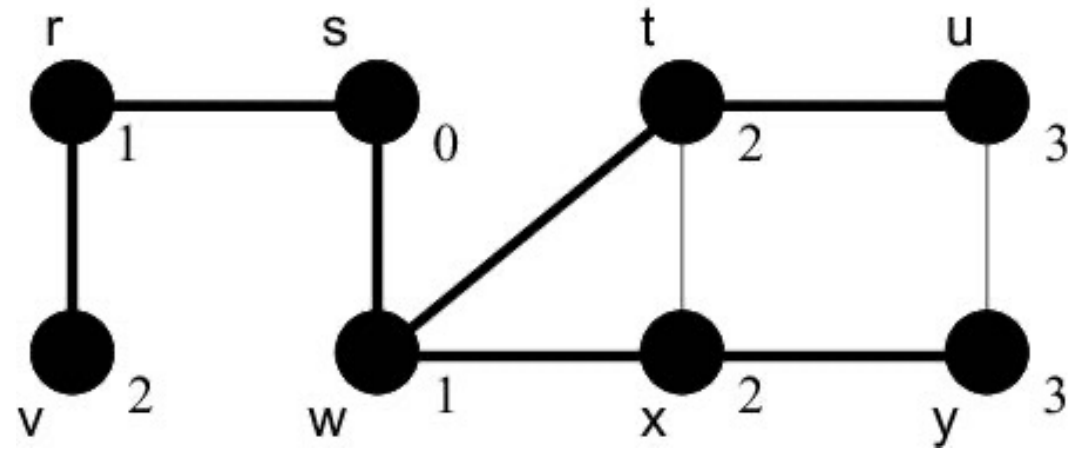
- aktuellen Knoten k aus Warteschlange entnehmen
- alle von k aus erreichbaren weißen Knoten:
 - grau färben
 - Entfernung ist Entfernungswert von k plus 1
 - in die Warteschlange aufnehmen
 - π Vorgänger im aufspannenden Baum auf k setzen
- k wird schwarz

Breitendurchlauf – Beispiel





Breitendurchlauf – Beispiel





Tiefendurchlauf

- DFS für **depth-first search** realisiert einen rekursiven Durchlauf durch einen gerichteten Graphen
- Farbmarkierungen für den Bearbeitungsstatus eines Knotens
 - Weiß markiert: noch nicht bearbeitete Knoten
 - graue Knoten: in Bearbeitung
 - schwarze: bereits fertig abgearbeitet



Tiefendurchlauf: Algorithmus

algorithm **DFS**(G)

Eingabe: ein Graph G

```
foreach Knoten u aus V[G] do
    farbe[u] := weiß;
    pi[u] := null
```

od

zeit := 0;

```
foreach Knoten u aus V [G] do
    if farbe[u] = weiß then
        DFSvisit(u)
```

fi

od

algorithm **DFSvisit**(u)

Eingabe: ein Knoten u

farbe[u] := grau; zeit := zeit+1;

d[u] := zeit;

```
foreach v in Zielknoten ausgehender Kanten von u do
    if farbe[v] = weiß then
        pi[v] := u;
        DFSvisit(v)
```

fi

od

farbe[u] := schwarz; zeit := zeit+1;

f[u] := zeit;



Tiefendurchlauf: Vorgehen

- rekursiver Abstieg
- pro Knoten zwei Werte plus Farbwert
 - Beginn der Bearbeitung d
 - Ende der Bearbeitung f
- rekursiver Aufruf nur bei weißen Knoten: Terminierung der Rekursion garantiert!
- Komplexität: $O(|V| + |E|)$, da DFS-visit für jeden Knoten genau einmal aufgerufen wird.
- Beim Aufruf werden alle von dem Knoten ausgehenden Kanten genau einmal besucht.

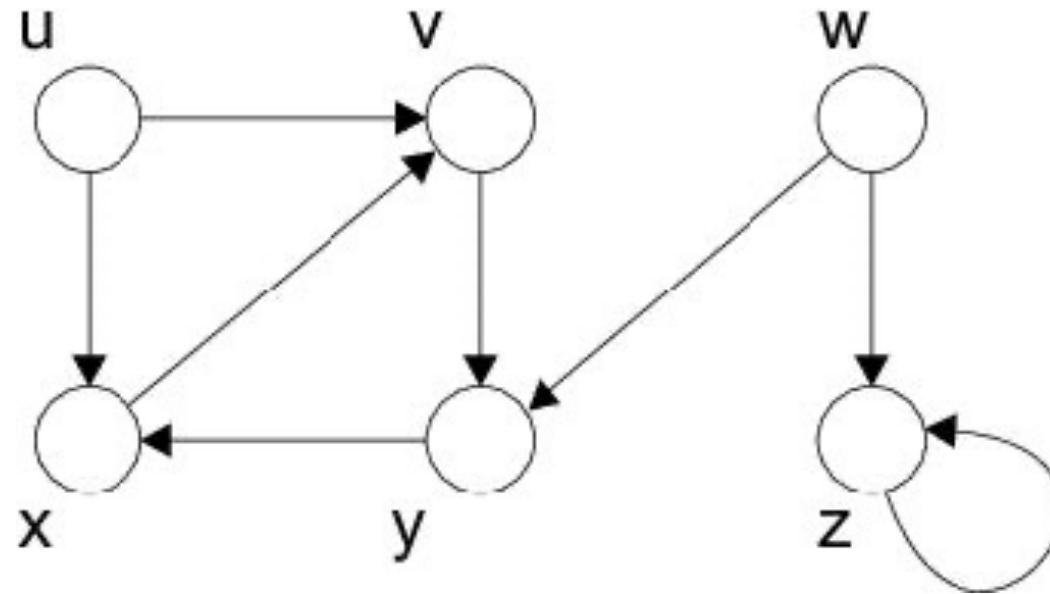


Tiefendurchlauf: Informationen über Kanten (u,v) beim Durchlauf

- Kanten des aufspannenden Baumes: Zielknoten bei Test ist weiß
- B Kanten: Zielknoten beim Test grau
 - Back-Edges oder Rückkanten im aufgespannten Baum
 - Eine mit B markierte Kante zeigt einen Zyklus an!
- F Kanten: beim Test wird ein schwarzer Knoten gefunden, dessen Bearbeitungsintervall ins Intervall des aktuell bearbeiten Knotens passt:
 - Forward-Edge bzw. Vorwärtskante in den aufgespannten Baum
 - „anderer Weg zu einem Nachkommen“
- schwarzer Zielknoten v, dessen Intervall nicht ins Intervall passt ($d[u] > d[v]$):
 - C für Cross-Edge, eine Kante, die zwei aufspannende Bäume verbindet

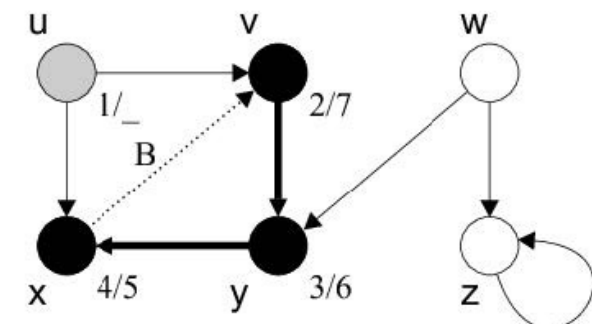
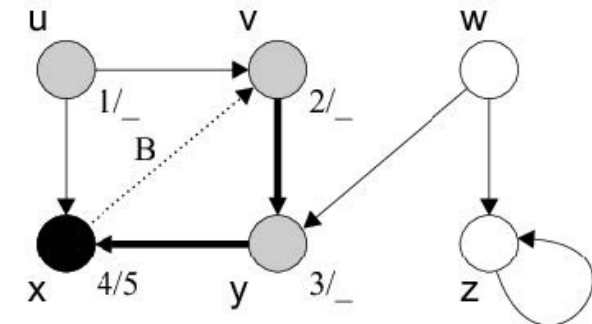
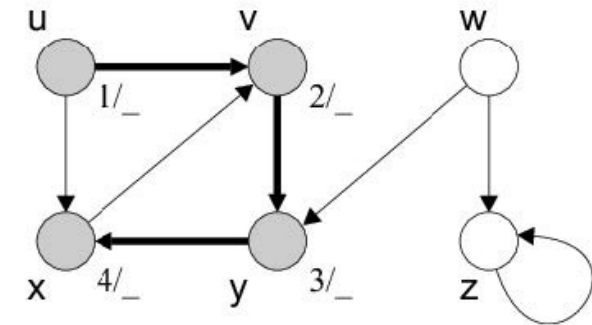
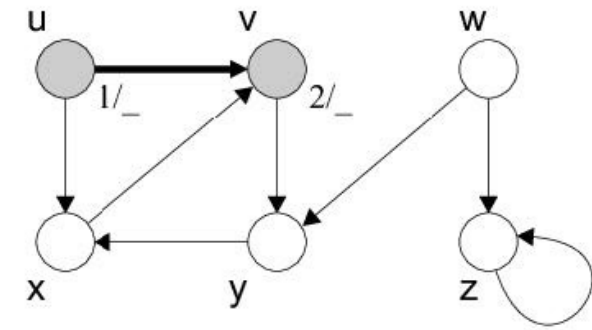
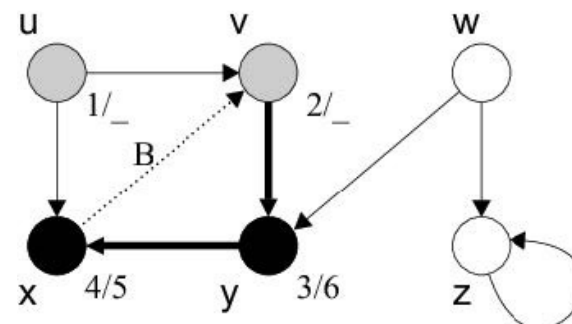
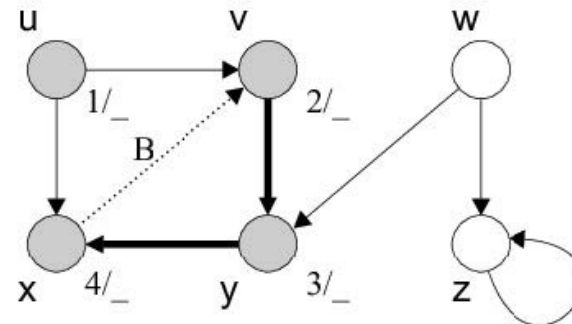
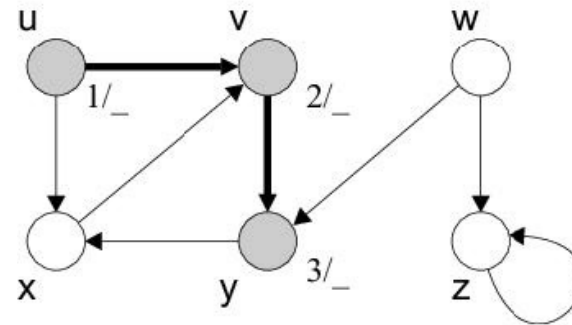
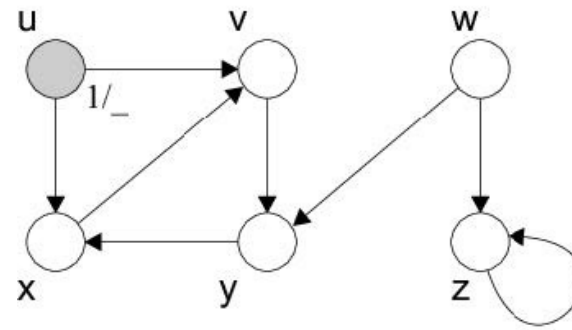


Tiefendurchlauf: Beispielgraph



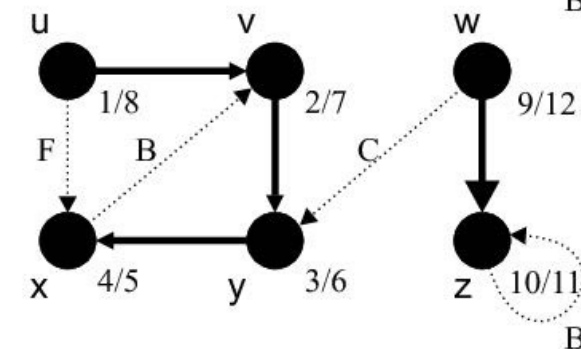
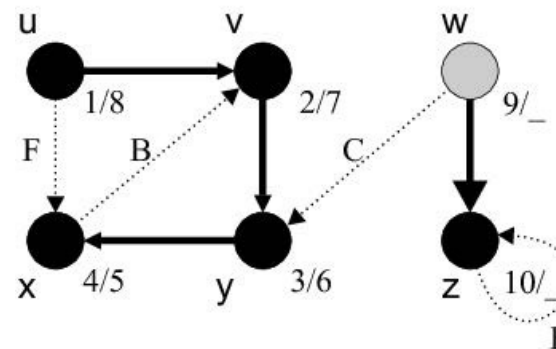
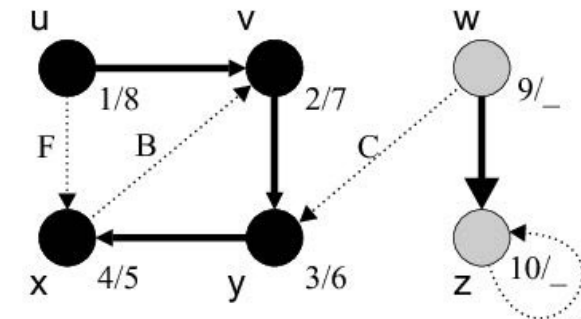
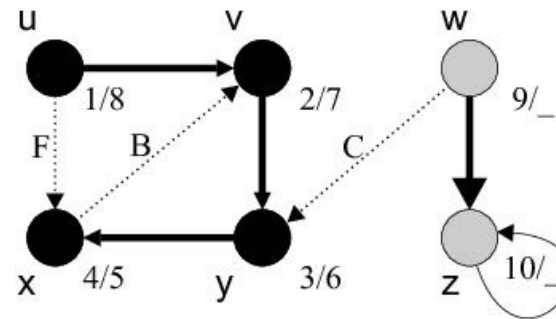
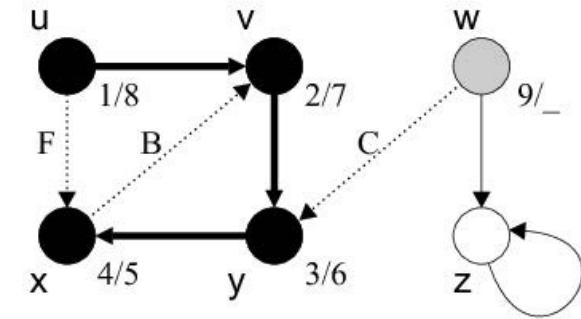
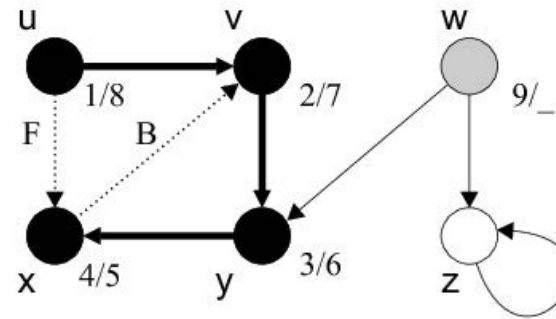
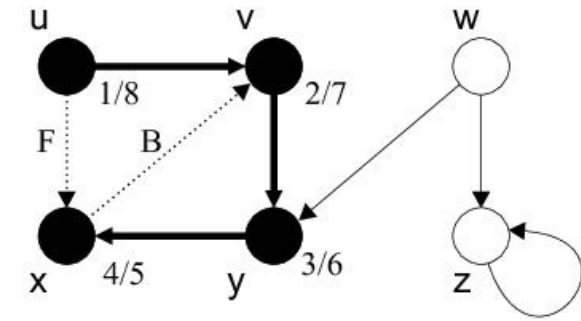
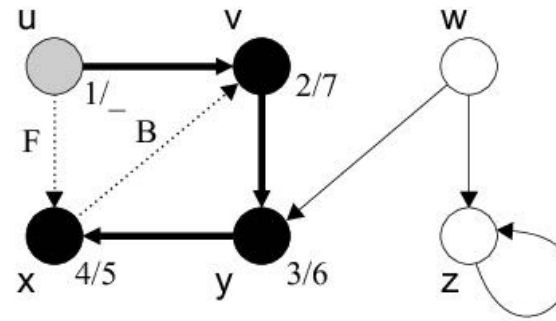


Tiefendurchlauf:





Tiefendurchlauf:





Einsatz von DFS

- Test auf Zyklenfreiheit
 - basiert auf dem Erkennen von Back-Edges
 - effizienter als beispielsweise Konstruktion der transitiven Hülle
- Topologisches Sortieren
 - „topologisch“ sortieren nach Nachbarschaft, nicht nach totaler Ordnung



Topologisches Sortieren

- gegeben: azyklischer gerichteter Graph
- gesucht: Reihenfolge der Knoten, so dass jeder Knoten nach all seinen Vorgängern kommt
- keine „Rückkanten“
- Scheduling bei kausalen / zeitlichen Abhängigkeiten
 - Projektmanagement
 - Anweisungen im Prozessor
- mathematisch: Konstruktion einer totalen Ordnung aus einer Halbordnung

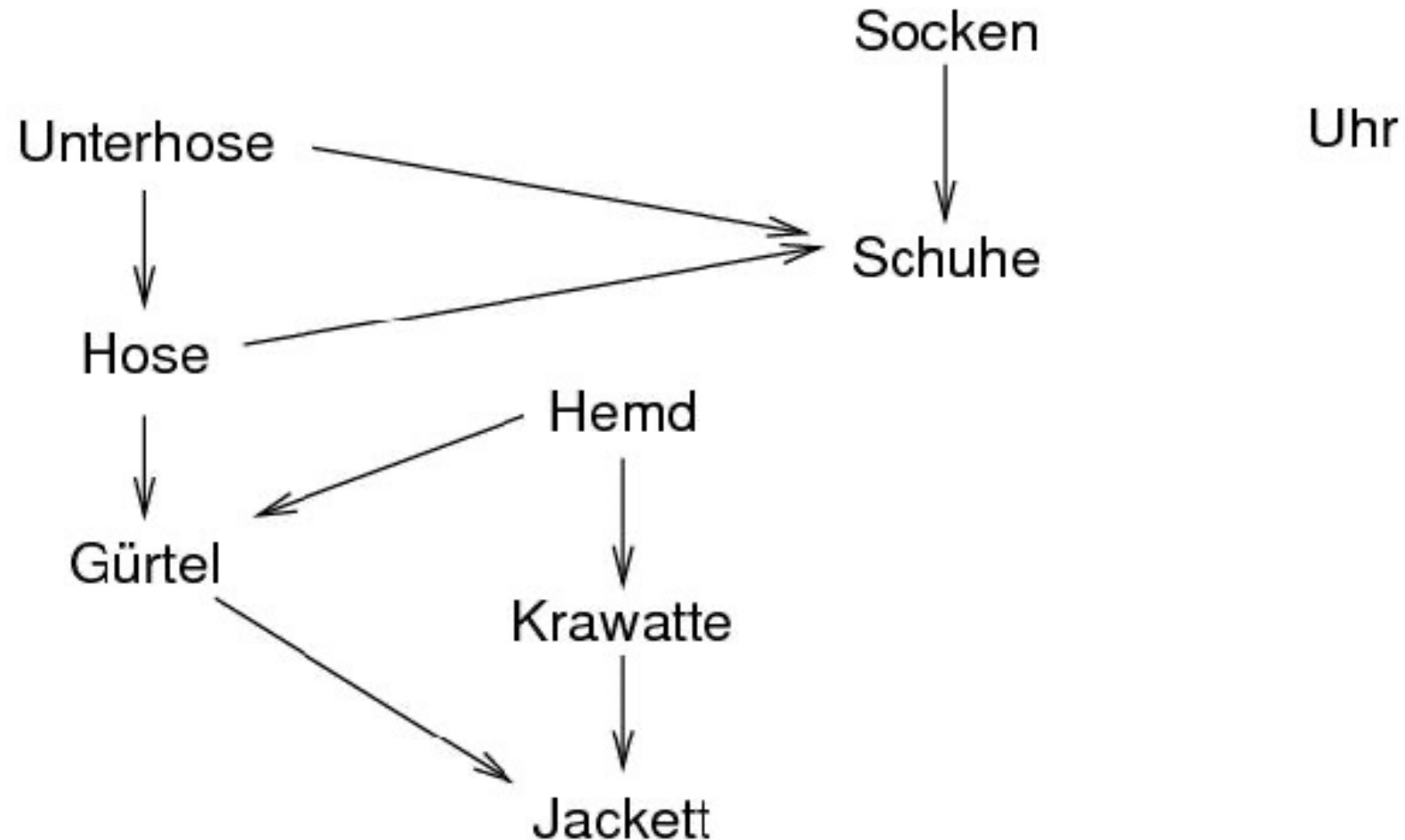


Der zerstreute Professor

- Ein zerstreuter Professor hat Probleme, morgens seine Kleidungsstücke in der richtigen Reihenfolge anzuziehen. Daher legt er die Reihenfolgebedingungen beim Ankleiden fest:
 - Unterhose vor Hose
 - Hose vor Gürtel
 - Hemd vor Gürtel
 - Gürtel vor Jackett
 - Hemd vor Krawatte
 - Krawatte vor Jackett
 - Socken vor Schuhen
 - Unterhose vor Schuhen
 - Hose vor Schuhen
 - Uhr: egal



Topologisches Sortieren: Vorgaben





Topologisches Sortieren und DFS

- DFS erstellt topologische Ordnung „on-the-fly“
- Sortierung nach f -Wert (invers) ergibt korrekte Reihenfolge
- statt expliziter Sortierung nach f : Knoten beim Setzen des f -Wertes vorne in eine verkettete Liste einhängen